МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний університет “Львівська політехніка”

Інститут комп’ютерних наук та інформаційних технологій

Кафедра автоматизованих систем управління

Звіт

про виконання лабораторної роботи №3

з дисципліни “ Паралельні обчислення і розподілені системи”

на тему:

“RESTful веб сервіси (частина 2)”

Виконав:

студент групи ОІ-21СП

Горбатюк Д. О.

Прийняла:

доцентка кафедри АСУ,

Федевич О. Ю.

Львів - 2025

Лабораторна робота №3

Тема: RESTful веб сервіси (частина 2).

Мета роботи: Оволодіти практичними прийомами розробки RESTful веб сервісів.

Завдання лабораторної роботи

Варіант по списку групи - 1

Завдання:

1. Система Факультатив. Існує перелік Курсів, за кожним з яких закріплений один Викладач. Студент записується на один або кілька Курсів, дані про реєстрацію зберігаються. Після завершення навчання Викладач виставляє Студенту оцінку, яка зберігається в Архіві.

ХІД РОБОТИ

1. Мовою програмування обрано Java, адже ця мова програмування має доволі зручний фреймворк Spring Boot, для створення подібних сервісів.
2. Створив проєкт у відповідному середовищі (див. Рис. 1.1):

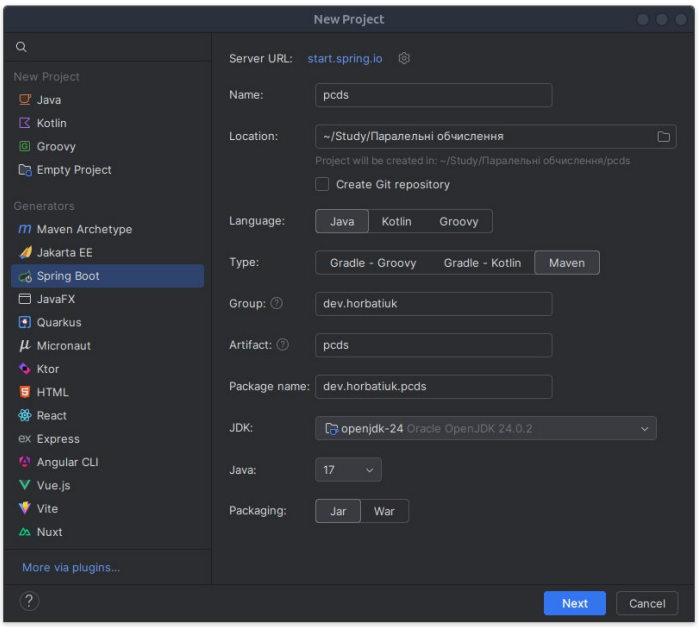
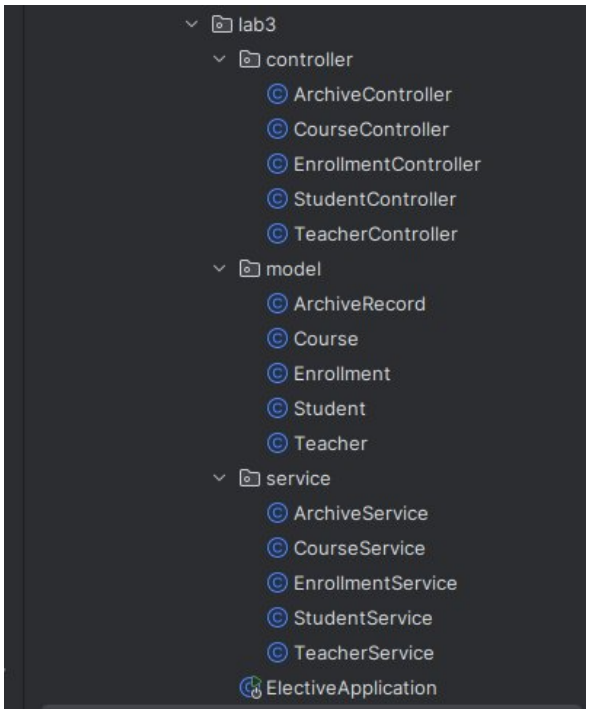


Рис. 1.1. Створення проєкту

3. Створив наступну архітектуру додатку:



4. Написав код всіх класів (див. Додатки 1-16).
5. Приклад виконання:

Блок 1 - файл lab3_test.sh, набір запитів до сервісу;
Блок 2 - відповідь на запит через консоль

```
BASE_URL="http://localhost:8080"

echo "==================================================="
echo "LAB TEST START: RESTful web service"
echo "==================================================="

echo
echo "1) CREATE TEACHER #1"
curl -s -X POST "$BASE_URL/teachers" \
-H "Content-Type: application/json" \
-d '{
  "fullName":"Ivan Petrenko",
  "department":"Computer Science",
  "email":"ivan.petrenko@university.edu"
}'
echo -e "\n"

echo "2) CREATE TEACHER #2"
curl -s -X POST "$BASE_URL/teachers" \
-H "Content-Type: application/json" \
-d '{
```

```
  "fullName":"Olena Shevchenko",
  "department":"Software Engineering",
  "email":"olena.shevchenko@university.edu"
}'
echo -e "\n"

echo "3) GET ALL TEACHERS"
curl -s "$BASE_URL/teachers"
echo -e "\n"

echo "4) GET TEACHER BY ID = 1"
curl -s "$BASE_URL/teachers/1"
echo -e "\n"

echo "5) UPDATE TEACHER ID = 1"
curl -s -X PUT "$BASE_URL/teachers/1" \
-H "Content-Type: application/json" \
-d '{
  "fullName":"Ivan Petrenko Updated",
  "department":"Distributed Systems",
  "email":"ivan.petrenko.updated@university.edu"
}'
echo -e "\n"

echo "6) GET UPDATED TEACHER ID = 1"
curl -s "$BASE_URL/teachers/1"
echo -e "\n"

echo "7) CREATE STUDENT #1"
curl -s -X POST "$BASE_URL/students" \
-H "Content-Type: application/json" \
-d '{
  "fullName":"Danylo Horbatiuk",
  "groupName":"OI-21SP",
  "email":"danylo@student.edu"
}'
echo -e "\n"

echo "8) CREATE STUDENT #2"
curl -s -X POST "$BASE_URL/students" \
-H "Content-Type: application/json" \
-d '{
  "fullName":"Andrii Melnyk",
  "groupName":"OI-21SP",
  "email":"andrii@student.edu"
}'
echo -e "\n"

echo "9) GET ALL STUDENTS"
curl -s "$BASE_URL/students"
echo -e "\n"
```

```
echo "10) GET STUDENT BY ID = 1"
curl -s "$BASE_URL/students/1"
echo -e "\n"

echo "11) UPDATE STUDENT ID = 1"
curl -s -X PUT "$BASE_URL/students/1" \
-H "Content-Type: application/json" \
-d '{
  "fullName":"Danylo Horbatiuk Updated",
  "groupName":"OI-22SP",
  "email":"danylo.updated@student.edu"
}'
echo -e "\n"

echo "12) GET UPDATED STUDENT ID = 1"
curl -s "$BASE_URL/students/1"
echo -e "\n"

echo "13) CREATE COURSE #1"
curl -s -X POST "$BASE_URL/courses" \
-H "Content-Type: application/json" \
-d '{
  "title":"Parallel Computing",
  "credits":5,
  "teacherId":1
}'
echo -e "\n"

echo "14) CREATE COURSE #2"
curl -s -X POST "$BASE_URL/courses" \
-H "Content-Type: application/json" \
-d '{
  "title":"Distributed Systems",
  "credits":4,
  "teacherId":2
}'
echo -e "\n"

echo "15) GET ALL COURSES"
curl -s "$BASE_URL/courses"
echo -e "\n"

echo "16) GET COURSE BY ID = 1"
curl -s "$BASE_URL/courses/1"
echo -e "\n"

echo "17) UPDATE COURSE ID = 1"
curl -s -X PUT "$BASE_URL/courses/1" \
-H "Content-Type: application/json" \
-d '{
  "title":"Parallel Computing Advanced",
  "credits":6,
```

```
  "teacherId":1
}'
echo -e "\n"

echo "18) GET UPDATED COURSE ID = 1"
curl -s "$BASE_URL/courses/1"
echo -e "\n"

echo "19) NEGATIVE TEST: CREATE COURSE WITH INVALID TEACHER"
curl -s -X POST "$BASE_URL/courses" \
-H "Content-Type: application/json" \
-d '{
  "title":"Invalid Course",
  "credits":3,
  "teacherId":99
}'
echo -e "\n"

echo "20) CREATE ENROLLMENT #1"
curl -s -X POST "$BASE_URL/enrollments" \
-H "Content-Type: application/json" \
-d '{
  "studentId":1,
  "courseId":1
}'
echo -e "\n"

echo "21) CREATE ENROLLMENT #2"
curl -s -X POST "$BASE_URL/enrollments" \
-H "Content-Type: application/json" \
-d '{
  "studentId":2,
  "courseId":2
}'
echo -e "\n"

echo "22) GET ALL ENROLLMENTS"
curl -s "$BASE_URL/enrollments"
echo -e "\n"

echo "23) GET ENROLLMENT BY ID = 1"
curl -s "$BASE_URL/enrollments/1"
echo -e "\n"

echo "24) UPDATE ENROLLMENT ID = 1"
curl -s -X PUT "$BASE_URL/enrollments/1" \
-H "Content-Type: application/json" \
-d '{
  "studentId":1,
  "courseId":2
}'
echo -e "\n"
```

```
echo "25) GET UPDATED ENROLLMENT ID = 1"
curl -s "$BASE_URL/enrollments/1"
echo -e "\n"

echo "26) NEGATIVE TEST: DUPLICATE ENROLLMENT"
curl -s -X POST "$BASE_URL/enrollments" \
-H "Content-Type: application/json" \
-d '{
  "studentId":1,
  "courseId":2
}'
echo -e "\n"

echo "27) NEGATIVE TEST: ENROLLMENT WITH INVALID STUDENT"
curl -s -X POST "$BASE_URL/enrollments" \
-H "Content-Type: application/json" \
-d '{
  "studentId":99,
  "courseId":1
}'
echo -e "\n"

echo "28) NEGATIVE TEST: ENROLLMENT WITH INVALID COURSE"
curl -s -X POST "$BASE_URL/enrollments" \
-H "Content-Type: application/json" \
-d '{
  "studentId":1,
  "courseId":99
}'
echo -e "\n"

echo "29) CREATE ARCHIVE RECORD #1"
curl -s -X POST "$BASE_URL/archive" \
-H "Content-Type: application/json" \
-d '{
  "studentId":1,
  "courseId":1,
  "teacherId":1,
  "grade":95
}'
echo -e "\n"

echo "30) CREATE ARCHIVE RECORD #2"
curl -s -X POST "$BASE_URL/archive" \
-H "Content-Type: application/json" \
-d '{
  "studentId":2,
  "courseId":2,
  "teacherId":2,
  "grade":88
}'
echo -e "\n"
```

```
echo "31) GET ALL ARCHIVE RECORDS"
curl -s "$BASE_URL/archive"
echo -e "\n"

echo "32) GET ARCHIVE RECORD BY ID = 1"
curl -s "$BASE_URL/archive/1"
echo -e "\n"

echo "33) UPDATE ARCHIVE RECORD ID = 1"
curl -s -X PUT "$BASE_URL/archive/1" \
-H "Content-Type: application/json" \
-d '{
  "studentId":1,
  "courseId":1,
  "teacherId":1,
  "grade":100
}'
echo -e "\n"

echo "34) GET UPDATED ARCHIVE RECORD ID = 1"
curl -s "$BASE_URL/archive/1"
echo -e "\n"

echo "35) NEGATIVE TEST: ARCHIVE INVALID GRADE"
curl -s -X POST "$BASE_URL/archive" \
-H "Content-Type: application/json" \
-d '{
  "studentId":1,
  "courseId":1,
  "teacherId":1,
  "grade":150
}'
echo -e "\n"

echo "36) NEGATIVE TEST: ARCHIVE INVALID STUDENT"
curl -s -X POST "$BASE_URL/archive" \
-H "Content-Type: application/json" \
-d '{
  "studentId":99,
  "courseId":1,
  "teacherId":1,
  "grade":90
}'
echo -e "\n"

echo "37) DELETE ENROLLMENT ID = 2"
curl -s -X DELETE "$BASE_URL/enrollments/2" -w "\nHTTP STATUS: %{http_code}\n"
echo

echo "38) DELETE ARCHIVE RECORD ID = 2"
curl -s -X DELETE "$BASE_URL/archive/2" -w "\nHTTP STATUS: %{http_code}\n"
```

```
echo

echo "39) DELETE COURSE ID = 2"
curl -s -X DELETE "$BASE_URL/courses/2" -w "\nHTTP STATUS: %{http_code}\n"
echo

echo "40) DELETE STUDENT ID = 2"
curl -s -X DELETE "$BASE_URL/students/2" -w "\nHTTP STATUS: %{http_code}\n"
echo

echo "41) DELETE TEACHER ID = 2"
curl -s -X DELETE "$BASE_URL/teachers/2" -w "\nHTTP STATUS: %{http_code}\n"
echo

echo "42) FINAL GET ALL TEACHERS"
curl -s "$BASE_URL/teachers"
echo -e "\n"

echo "43) FINAL GET ALL STUDENTS"
curl -s "$BASE_URL/students"
echo -e "\n"

echo "44) FINAL GET ALL COURSES"
curl -s "$BASE_URL/courses"
echo -e "\n"

echo "45) FINAL GET ALL ENROLLMENTS"
curl -s "$BASE_URL/enrollments"
echo -e "\n"

echo "46) FINAL GET ALL ARCHIVE RECORDS"
curl -s "$BASE_URL/archive"
echo -e "\n"
```

```
==================================================
LAB TEST START: RESTful web service
==================================================

1) CREATE TEACHER #1
{"id":1,"fullName":"Ivan Petrenko","department":"Computer
Science","email":"ivan.petrenko@university.edu"}

2) CREATE TEACHER #2
{"id":2,"fullName":"Olena Shevchenko","department":"Software
Engineering","email":"olena.shevchenko@university.edu"}

3) GET ALL TEACHERS
[{"id":1,"fullName":"Ivan Petrenko","department":"Computer
Science","email":"ivan.petrenko@university.edu"},{"id":2,"fullName":"Olena
Shevchenko","department":"Software
```

Engineering","email":"olena.shevchenko@university.edu"}]

4) GET TEACHER BY ID = 1
{"id":1,"fullName":"Ivan Petrenko","department":"Computer Science","email":"ivan.petrenko@university.edu"}

5) UPDATE TEACHER ID = 1
{"id":1,"fullName":"Ivan Petrenko Updated","department":"Distributed Systems","email":"ivan.petrenko.updated@university.edu"}

6) GET UPDATED TEACHER ID = 1
{"id":1,"fullName":"Ivan Petrenko Updated","department":"Distributed Systems","email":"ivan.petrenko.updated@university.edu"}

7) CREATE STUDENT #1
{"id":1,"fullName":"Danylo Horbatiuk","groupName":"OI-21SP","email":"danylo@student.edu"}

8) CREATE STUDENT #2
{"id":2,"fullName":"Andrii Melnyk","groupName":"OI-21SP","email":"andrii@student.edu"}

9) GET ALL STUDENTS
[{"id":1,"fullName":"Danylo Horbatiuk","groupName":"OI-21SP","email":"danylo@student.edu"},{"id":2,"fullName":"Andrii Melnyk","groupName":"OI-21SP","email":"andrii@student.edu"}]

10) GET STUDENT BY ID = 1
{"id":1,"fullName":"Danylo Horbatiuk","groupName":"OI-21SP","email":"danylo@student.edu"}

11) UPDATE STUDENT ID = 1
{"id":1,"fullName":"Danylo Horbatiuk Updated","groupName":"OI-22SP","email":"danylo.updated@student.edu"}

12) GET UPDATED STUDENT ID = 1
{"id":1,"fullName":"Danylo Horbatiuk Updated","groupName":"OI-22SP","email":"danylo.updated@student.edu"}

13) CREATE COURSE #1
{"id":1,"name":null,"description":null,"credits":5,"teacherId":1}

14) CREATE COURSE #2
{"id":2,"name":null,"description":null,"credits":4,"teacherId":2}

15) GET ALL COURSES
[{"id":1,"name":null,"description":null,"credits":5,"teacherId":1},{"id":2,"name":null,"description":null,"credits":4,"teacherId":2}]

16) GET COURSE BY ID = 1
{"id":1,"name":null,"description":null,"credits":5,"teacherId":1}

```
17) UPDATE COURSE ID = 1
{"id":1,"name":null,"description":null,"credits":6,"teacherId":1}

18) GET UPDATED COURSE ID = 1
{"id":1,"name":null,"description":null,"credits":6,"teacherId":1}

19) NEGATIVE TEST: CREATE COURSE WITH INVALID TEACHER
Teacher with given id does not exist

20) CREATE ENROLLMENT #1
{"id":1,"studentId":1,"courseId":1,"enrollmentDate":null,"status":null}

21) CREATE ENROLLMENT #2
{"id":2,"studentId":2,"courseId":2,"enrollmentDate":null,"status":null}

22) GET ALL ENROLLMENTS
[{"id":1,"studentId":1,"courseId":1,"enrollmentDate":null,"status":null},{"id":2
,"studentId":2,"courseId":2,"enrollmentDate":null,"status":null}]

23) GET ENROLLMENT BY ID = 1
{"id":1,"studentId":1,"courseId":1,"enrollmentDate":null,"status":null}

24) UPDATE ENROLLMENT ID = 1
{"id":1,"studentId":1,"courseId":2,"enrollmentDate":null,"status":null}

25) GET UPDATED ENROLLMENT ID = 1
{"id":1,"studentId":1,"courseId":2,"enrollmentDate":null,"status":null}

26) NEGATIVE TEST: DUPLICATE ENROLLMENT
Student is already enrolled in this course

27) NEGATIVE TEST: ENROLLMENT WITH INVALID STUDENT
Student with given id does not exist

28) NEGATIVE TEST: ENROLLMENT WITH INVALID COURSE
Course with given id does not exist

29) CREATE ARCHIVE RECORD #1
{"id":1,"studentId":1,"courseId":1,"teacherId":1,"grade":95,"completedAt":null}

30) CREATE ARCHIVE RECORD #2
{"id":2,"studentId":2,"courseId":2,"teacherId":2,"grade":88,"completedAt":null}

31) GET ALL ARCHIVE RECORDS
[{"id":1,"studentId":1,"courseId":1,"teacherId":1,"grade":95,"completedAt":null}
,{"id":2,"studentId":2,"courseId":2,"teacherId":2,"grade":88,"completedAt":null}
]

32) GET ARCHIVE RECORD BY ID = 1
```

```
{"id":1,"studentId":1,"courseId":1,"teacherId":1,"grade":95,"completedAt":null}

33) UPDATE ARCHIVE RECORD ID = 1
{"id":1,"studentId":1,"courseId":1,"teacherId":1,"grade":100,"completedAt":null}

34) GET UPDATED ARCHIVE RECORD ID = 1
{"id":1,"studentId":1,"courseId":1,"teacherId":1,"grade":100,"completedAt":null}

35) NEGATIVE TEST: ARCHIVE INVALID GRADE
Grade must be in range 0..100

36) NEGATIVE TEST: ARCHIVE INVALID STUDENT
Student with given id does not exist

37) DELETE ENROLLMENT ID = 2

HTTP STATUS: 204

38) DELETE ARCHIVE RECORD ID = 2

HTTP STATUS: 204

39) DELETE COURSE ID = 2

HTTP STATUS: 204

40) DELETE STUDENT ID = 2

HTTP STATUS: 204

41) DELETE TEACHER ID = 2

HTTP STATUS: 204

42) FINAL GET ALL TEACHERS
[{"id":1,"fullName":"Ivan Petrenko Updated","department":"Distributed
Systems","email":"ivan.petrenko.updated@university.edu"}]

43) FINAL GET ALL STUDENTS
[{"id":1,"fullName":"Danylo Horbatiuk Updated","groupName":"OI-
22SP","email":"danylo.updated@student.edu"}]

44) FINAL GET ALL COURSES
[{"id":1,"name":null,"description":null,"credits":6,"teacherId":1}]

45) FINAL GET ALL ENROLLMENTS
[{"id":1,"studentId":1,"courseId":2,"enrollmentDate":null,"status":null}]

46) FINAL GET ALL ARCHIVE RECORDS
```

```
[{"id":1,"studentId":1,"courseId":1,"teacherId":1,"grade":100,"completedAt":null
}]
```


6. REST-сервіс передбачає наступні ендпоінти:

REST endpoints

Students:

GET /students — отримати всіх студентів

GET /students/{id} — отримати студента за id

POST /students — створити студента

PUT /students/{id} — оновити студента

DELETE /students/{id} — видалити студента

Teachers:

GET /teachers — отримати всіх викладачів

GET /teachers/{id} — отримати викладача за id

POST /teachers — створити викладача

PUT /teachers/{id} — оновити викладача

DELETE /teachers/{id} — видалити викладача

Courses:

GET /courses — отримати всі курси

GET /courses/{id} — отримати курс за id

POST /courses — створити курс

PUT /courses/{id} — оновити курс

DELETE /courses/{id} — видалити курс

Enrollments:

GET /enrollments — отримати всі записи зарахування

GET /enrollments/{id} — отримати запис за id

POST /enrollments — створити запис зарахування

PUT /enrollments/{id} — оновити запис зарахування

DELETE /enrollments/{id} — видалити запис зарахування

Archive:

GET /archive — отримати всі архівні записи

GET /archive/{id} — отримати архівний запис за id

POST /archive — створити архівний запис

PUT /archive/{id} — оновити архівний запис

DELETE /archive/{id} — видалити архівний запис

ВИСНОВКИ

У ході лабораторної роботи я реалізував REST-сервіс мовою Java за обраною доменною областю: організація заходів, створення команд і квестів. Перелік створених ендпоінтів наведений вище.

Тестування ендпоінтів відбувалось за допомогою curl і термінал, що є доволі швидким рішенням, для тестування, яке не вимагає встановлення додаткового ПЗ.

Ця лабораторна робота справді показала, що RESTful сервіси виступають базовим способом взаємодії між окремими компонентами розподілених систем. Саме через подібні API різні модулі, сервери або мікросервіси можуть обмінюватися даними між собою незалежно від платформи чи мови реалізації.

ДОДАТКИ

Додаток 1

Код файлу ArchiveController.java

```
package dev.horbatiuk.pcds.lab3.controller;

import dev.horbatiuk.pcds.lab3.model.ArchiveRecord;
import dev.horbatiuk.pcds.lab3.service.ArchiveService;
import dev.horbatiuk.pcds.lab3.service.CourseService;
import dev.horbatiuk.pcds.lab3.service.StudentService;
import dev.horbatiuk.pcds.lab3.service.TeacherService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/archive")
public class ArchiveController {
    private final ArchiveService archiveService;
    private final StudentService studentService;
    private final CourseService courseService;
    private final TeacherService teacherService;

    public ArchiveController(ArchiveService archiveService,
                             StudentService studentService,
                             CourseService courseService,
                             TeacherService teacherService) {
        this.archiveService = archiveService;
        this.studentService = studentService;
        this.courseService = courseService;
        this.teacherService = teacherService;
    }

    @GetMapping
    public List<ArchiveRecord> getAll() {
        return archiveService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<ArchiveRecord> getById(@PathVariable Long id) {
        ArchiveRecord record = archiveService.getById(id);
        return record != null ? ResponseEntity.ok(record) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<?> create(@RequestBody ArchiveRecord record) {
        if (!studentService.exists(record.getStudentId())) {
            return ResponseEntity.badRequest().body("Student with given id does
not exist");
```

```
        }
        if (!courseService.exists(record.getCourseId())) {
            return ResponseEntity.badRequest().body("Course with given id does not
exist");
        }
        if (!teacherService.exists(record.getTeacherId())) {
            return ResponseEntity.badRequest().body("Teacher with given id does
not exist");
        }
        if (record.getGrade() == null || record.getGrade() < 0 ||
record.getGrade() > 100) {
            return ResponseEntity.badRequest().body("Grade must be in range
0..100");
        }
        return ResponseEntity.ok(archiveService.create(record));
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> update(@PathVariable Long id, @RequestBody
ArchiveRecord record) {
        if (!studentService.exists(record.getStudentId())) {
            return ResponseEntity.badRequest().body("Student with given id does
not exist");
        }
        if (!courseService.exists(record.getCourseId())) {
            return ResponseEntity.badRequest().body("Course with given id does not
exist");
        }
        if (!teacherService.exists(record.getTeacherId())) {
            return ResponseEntity.badRequest().body("Teacher with given id does
not exist");
        }
        if (record.getGrade() == null || record.getGrade() < 0 ||
record.getGrade() > 100) {
            return ResponseEntity.badRequest().body("Grade must be in range
0..100");
        }
        ArchiveRecord updated = archiveService.update(id, record);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return archiveService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
}
```

Додаток 2

Код файлу CourseController.java

```
package dev.horbatiuk.pcds.lab3.controller;

import dev.horbatiuk.pcds.lab3.model.Course;
import dev.horbatiuk.pcds.lab3.service.CourseService;
import dev.horbatiuk.pcds.lab3.service.TeacherService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/courses")
public class CourseController {
    private final CourseService courseService;
    private final TeacherService teacherService;

    public CourseController(CourseService courseService, TeacherService
teacherService) {
        this.courseService = courseService;
        this.teacherService = teacherService;
    }

    @GetMapping
    public List<Course> getAll() {
        return courseService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Course> getById(@PathVariable Long id) {
        Course course = courseService.getById(id);
        return course != null ? ResponseEntity.ok(course) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<?> create(@RequestBody Course course) {
        if (!teacherService.exists(course.getTeacherId())) {
            return ResponseEntity.badRequest().body("Teacher with given id does
not exist");
        }
        return ResponseEntity.ok(courseService.create(course));
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> update(@PathVariable Long id, @RequestBody Course
course) {
        if (!teacherService.exists(course.getTeacherId())) {
```

```
            return ResponseEntity.badRequest().body("Teacher with given id does
not exist");
        }
        Course updated = courseService.update(id, course);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return courseService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
}
```

Додаток 3

Код файлу EnrollmentController.java

```
package dev.horbatiuk.pcds.lab3.controller;

import dev.horbatiuk.pcds.lab3.model.Enrollment;
import dev.horbatiuk.pcds.lab3.service.CourseService;
import dev.horbatiuk.pcds.lab3.service.EnrollmentService;
import dev.horbatiuk.pcds.lab3.service.StudentService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/enrollments")
public class EnrollmentController {
    private final EnrollmentService enrollmentService;
    private final StudentService studentService;
    private final CourseService courseService;

    public EnrollmentController(EnrollmentService enrollmentService,
                                StudentService studentService,
                                CourseService courseService) {
        this.enrollmentService = enrollmentService;
        this.studentService = studentService;
        this.courseService = courseService;
    }

    @GetMapping
    public List<Enrollment> getAll() {
        return enrollmentService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Enrollment> getById(@PathVariable Long id) {
        Enrollment enrollment = enrollmentService.getById(id);
        return enrollment != null ? ResponseEntity.ok(enrollment) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<?> create(@RequestBody Enrollment enrollment) {
        if (!studentService.exists(enrollment.getStudentId())) {
            return ResponseEntity.badRequest().body("Student with given id does
not exist");
        }
        if (!courseService.exists(enrollment.getCourseId())) {
            return ResponseEntity.badRequest().body("Course with given id does not
exist");
        }
```

```
        if (enrollmentService.isStudentAlreadyEnrolled(enrollment.getStudentId(),
enrollment.getCourseId())) {
            return ResponseEntity.badRequest().body("Student is already enrolled
in this course");
        }
        return ResponseEntity.ok(enrollmentService.create(enrollment));
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> update(@PathVariable Long id, @RequestBody Enrollment
enrollment) {
        if (!studentService.exists(enrollment.getStudentId())) {
            return ResponseEntity.badRequest().body("Student with given id does
not exist");
        }
        if (!courseService.exists(enrollment.getCourseId())) {
            return ResponseEntity.badRequest().body("Course with given id does not
exist");
        }
        Enrollment updated = enrollmentService.update(id, enrollment);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return enrollmentService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
}
```

Додаток 4

Код файлу StudentController.java

```
package dev.horbatiuk.pcds.lab3.controller;

import dev.horbatiuk.pcds.lab3.model.Student;
import dev.horbatiuk.pcds.lab3.service.StudentService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/students")
public class StudentController {
    private final StudentService studentService;

    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @GetMapping
    public List<Student> getAll() {
        return studentService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Student> getById(@PathVariable Long id) {
        Student student = studentService.getById(id);
        return student != null ? ResponseEntity.ok(student) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<Student> create(@RequestBody Student student) {
        return ResponseEntity.ok(studentService.create(student));
    }

    @PutMapping("/{id}")
    public ResponseEntity<Student> update(@PathVariable Long id, @RequestBody
Student student) {
        Student updated = studentService.update(id, student);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return studentService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
```

}

Додаток 5

Код файлу TeacherController.java

```
package dev.horbatiuk.pcds.lab3.controller;

import dev.horbatiuk.pcds.lab3.model.Teacher;
import dev.horbatiuk.pcds.lab3.service.TeacherService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/teachers")
public class TeacherController {
    private final TeacherService teacherService;

    public TeacherController(TeacherService teacherService) {
        this.teacherService = teacherService;
    }

    @GetMapping
    public List<Teacher> getAll() {
        return teacherService.getAll();
    }

    @GetMapping("/{id}")
    public ResponseEntity<Teacher> getById(@PathVariable Long id) {
        Teacher teacher = teacherService.getById(id);
        return teacher != null ? ResponseEntity.ok(teacher) :
ResponseEntity.notFound().build();
    }

    @PostMapping
    public ResponseEntity<Teacher> create(@RequestBody Teacher teacher) {
        return ResponseEntity.ok(teacherService.create(teacher));
    }
    @PutMapping("/{id}")
    public ResponseEntity<Teacher> update(@PathVariable Long id, @RequestBody
Teacher teacher) {
        Teacher updated = teacherService.update(id, teacher);
        return updated != null ? ResponseEntity.ok(updated) :
ResponseEntity.notFound().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        return teacherService.delete(id) ? ResponseEntity.noContent().build() :
ResponseEntity.notFound().build();
    }
}
```

Додаток 6

Код файлу ArchiveRecord.java

```
package dev.horbatiuk.pcds.lab3.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class ArchiveRecord {
    private Long id;
    private Long studentId;
    private Long courseId;
    private Long teacherId;
    private Integer grade;
    private String completedAt;
}
```

Додаток 7

Код файлу Course.java

```
package dev.horbatiuk.pcds.lab3.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class Course {
    private Long id;
    private String name;
    private String description;
    private Integer credits;
    private Long teacherId;
}
```

Додаток 8

Код файлу Enrollment.java

```
package dev.horbatiuk.pcds.lab3.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
```

```
@NoArgsConstructor
public class Enrollment {
    private Long id;
    private Long studentId;
    private Long courseId;
    private String enrollmentDate;
    private String status;
}
```

Додаток 9

Код файлу Student.java

```
package dev.horbatiuk.pcds.lab3.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class Student {
    private Long id;
    private String fullName;
    private String groupName;
    private String email;
}
```

Додаток 10

Код файлу Student.java

```
package dev.horbatiuk.pcds.lab3.model;

import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor
public class Teacher {
    private Long id;
    private String fullName;
    private String department;
    private String email;
}
```

Додаток 11

Код файлу ArchiveService.java

```
package dev.horbatiuk.pcds.lab3.service;

import dev.horbatiuk.pcds.lab3.model.ArchiveRecord;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class ArchiveService {
    private final Map<Long, ArchiveRecord> archive = new LinkedHashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<ArchiveRecord> getAll() {
        return new ArrayList<>(archive.values());
    }

    public ArchiveRecord getById(Long id) {
        return archive.get(id);
    }

    public ArchiveRecord create(ArchiveRecord record) {
        long id = counter.getAndIncrement();
        record.setId(id);
        archive.put(id, record);
        return record;
    }

    public ArchiveRecord update(Long id, ArchiveRecord record) {
        if (!archive.containsKey(id)) {
            return null;
        }
        record.setId(id);
        archive.put(id, record);
        return record;
    }

    public boolean delete(Long id) {
        return archive.remove(id) != null;
    }
}
```

Додаток 12

Код файлу CourseService.java

```
package dev.horbatiuk.pcds.lab3.service;

import dev.horbatiuk.pcds.lab3.model.Course;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class CourseService {
    private final Map<Long, Course> courses = new LinkedHashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Course> getAll() {
        return new ArrayList<>(courses.values());
    }

    public Course getById(Long id) {
        return courses.get(id);
    }

    public Course create(Course course) {
        long id = counter.getAndIncrement();
        course.setId(id);
        courses.put(id, course);
        return course;
    }

    public Course update(Long id, Course course) {
        if (!courses.containsKey(id)) {
            return null;
        }
        course.setId(id);
        courses.put(id, course);
        return course;
    }

    public boolean delete(Long id) {
        return courses.remove(id) != null;
    }

    public boolean exists(Long id) {
        return courses.containsKey(id);
    }
}
```

Додаток 13

Код файлу EnrollmentService.java

```
package dev.horbatiuk.pcds.lab3.service;

import dev.horbatiuk.pcds.lab3.model.Enrollment;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class EnrollmentService {
    private final Map<Long, Enrollment> enrollments = new LinkedHashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Enrollment> getAll() {
        return new ArrayList<>(enrollments.values());
    }
    public Enrollment getById(Long id) {
        return enrollments.get(id);
    }
    public Enrollment create(Enrollment enrollment) {
        long id = counter.getAndIncrement();
        enrollment.setId(id);
        enrollments.put(id, enrollment);
        return enrollment;
    }
    public Enrollment update(Long id, Enrollment enrollment) {
        if (!enrollments.containsKey(id)) {
            return null;
        }
        enrollment.setId(id);
        enrollments.put(id, enrollment);
        return enrollment;
    }
    public boolean delete(Long id) {
        return enrollments.remove(id) != null;
    }
    public boolean exists(Long id) {
        return enrollments.containsKey(id);
    }
    public boolean isStudentAlreadyEnrolled(Long studentId, Long courseId) {
        return enrollments.values().stream()
                .anyMatch(e -> e.getStudentId().equals(studentId) &&
e.getCourseId().equals(courseId));
    }
}
```

Додаток 14

Код файлу StudentService.java

```
package dev.horbatiuk.pcds.lab3.service;

import dev.horbatiuk.pcds.lab3.model.Student;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class StudentService {
    private final Map<Long, Student> students = new LinkedHashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Student> getAll() {
        return new ArrayList<>(students.values());
    }

    public Student getById(Long id) {
        return students.get(id);
    }

    public Student create(Student student) {
        long id = counter.getAndIncrement();
        student.setId(id);
        students.put(id, student);
        return student;
    }

    public Student update(Long id, Student student) {
        if (!students.containsKey(id)) {
            return null;
        }
        student.setId(id);
        students.put(id, student);
        return student;
    }

    public boolean delete(Long id) {
        return students.remove(id) != null;
    }
    public boolean exists(Long id) {
        return students.containsKey(id);
    }
}
```

Додаток 15

Код файлу TeacherService.java

```
package dev.horbatiuk.pcds.lab3.service;

import dev.horbatiuk.pcds.lab3.model.Teacher;
import org.springframework.stereotype.Service;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicLong;

@Service
public class TeacherService {
    private final Map<Long, Teacher> teachers = new LinkedHashMap<>();
    private final AtomicLong counter = new AtomicLong(1);

    public List<Teacher> getAll() {
        return new ArrayList<>(teachers.values());
    }

    public Teacher getById(Long id) {
        return teachers.get(id);
    }

    public Teacher create(Teacher teacher) {
        long id = counter.getAndIncrement();
        teacher.setId(id);
        teachers.put(id, teacher);
        return teacher;
    }

    public Teacher update(Long id, Teacher teacher) {
        if (!teachers.containsKey(id)) {
            return null;
        }
        teacher.setId(id);
        teachers.put(id, teacher);
        return teacher;
    }

    public boolean delete(Long id) {
        return teachers.remove(id) != null;
    }
    public boolean exists(Long id) {
        return teachers.containsKey(id);
    }
}
```

Додаток 16

Код файлу ElectiveApplication.java

```
package dev.horbatiuk.pcds.lab3;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ElectiveApplication {
    public static void main(String[] args) {
        SpringApplication.run(ElectiveApplication.class, args);
    }
}
```